Recommender Systems

- Knowledge-based Recommender Systems -

https://youtu.be/RKXGH46AICs

Institute for Software Technology
Inffeldgasse 16b/2
A-8010 Graz
Austria

Goal

- ☞ Basic introduction into the concepts of knowledge-based recommender systems.

Why do we need Knowledge-based Recommendation?

(Often complex) products (items) with a low number of available ratings (e.g., cars, apartments, financial services)







Time span plays an important role

- five-year-old ratings for computers
- user lifestyle or family situation changes

Customers want to define their requirements explicitly

- different requirement dimensions (e.g., price, color, power, …)
- "the color of the car should be black“
- Often “dialog-based” (conversational)

Knowledge-based Recommendation Approaches

Constraint-based

- based on explicitly defined set of recommendation rules (constraints)
- retrieve items that fulfill recommendation rules and user requirements

Case-based

- based on different types of similarity measures
- retrieve items that are similar to the given user requirements

Both approaches are similar in their conversational recommendation process

- users specify the requirements
- recommender system tries to identify solutions
- if no solution can be found, users change their requirements

Constraint-based Recommendation

Knowledge base

- usually mediates between user requirements and item properties
- variables
  - user model features (requirements), item features (catalogue)
- set of constraints
  - logical implications (IF user requires A THEN proposed item should possess feature B)
  - hard and soft/weighted constraints

Derive a set of recommendable items

- items fulfill requirements and constraints (in the knowledge base)
- explanations – transparent line of reasoning
  - why this recommendation?
  - why no solution could be found and how to deal with this situation?

Constraint-based Recommendation Problem

Select items from this catalog that match the user's requirements

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

User's requirements can, for example, be

- "the price should be lower than 300 €"
- "the camera should be suited for sports photography"

Constraint-based Recommendation Problem (defined as CSP)

A knowledge-based recommendation problem can be defined as a Constraint Satisfaction Problem (CSP):

$$CSP(X_I \cup X_U, D, R \cup KB \cup I)$$

Definition:

- X_I, X_U: variables describing product and user model with domain D
- KB: knowledge base with constraints (e.g. **if** purpose=*on travel* **then** lower focal length < *28mm*)
- R: specific requirements of a user (e.g. purpose = *on travel*)
- I: product catalog

Solution:

- complete assignment to X_I and X_U (consistent with KB)

CSP Example

- $V = \{color, type, price\}$ /* $V = X_I \cup X_U$ */
- $D = \{dom(Color) = \{red, green, yellow\},$
  $dom(Type) = \{racing, city, mountain\},$
  $dom(Price) = \{300, 500, 800, 1000, 1200\}\}$
  /* Domain definitions for variables in V */
- $C_{\mathrm{KB}} = \{\{Color = red \wedge Type = racing \wedge Price = 1000\} \vee$
  $\{Color = green \wedge Type = city \wedge Price = 500\} \vee$
  $\{Color = red \wedge Type = city \wedge Price = 300\} \vee$
  $\{Color = yellow \wedge Type = city \wedge Price = 800\} \vee$
  $\{Color = red \wedge Type = mountain \wedge Price = 1200\}\}$
  /* Simple KB (incl. I) without additional constraints */
- $C_{\mathrm{R}} = \{Color = red \wedge Type = mountain \wedge Price < 500\}$
  /* User requirements R */

Conjunctive Query

Different from a constraint solver

- goal is not to find valid instantiations for CSP variables but to identify tuples (items) that fulfill the criteria of the query

Conjunctive query is executed on the item catalog

- a conjunctive database query
- a set of selection criteria that are connected conjunctively

σ[criteria](P)

- P*:* product assortment
- example: σ[mpix≥10, price<300](P) = {p4, p7}

Interacting with Constraint-based Recommenders

The user specifies his or her initial preferences

- all at once or
- Incrementally, for example, on question per page
- interactive dialog

The user is presented with a set of matching items

- with explanations as to why a certain item was recommended

The user might revise his or her requirements

- see alternative solutions
- narrow down the number of matching items

Interacting with Constraint-based Recommenders: Defaults

Support customers to choose a reasonable alternative

- unsure about which option to select
- simply do not know technical details

Type of defaults

- static defaults (e.g., OS-Installation = “Linux”)
- dependent defaults (e.g., if country = “Austria” then language = “German”)
- derived defaults (e.g., based on collaborative approaches)

Selecting the next question

- most users are not interested in specifying values for all properties
- identify properties that may be interesting for the user

Derived Defaults: Example

- **Interaction log: user item selections**

session	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
1	148	8.0	4×	2.5	no	no	yes
2	182	8.0	5×	2.7	yes	yes	no
3	148	8.0	4×	2.5	no	no	yes
4	182	8.0	5×	2.7	yes	yes	no
5	182	8.0	5×	2.7	yes	yes	no
6	199	9.0	3×	3.0	yes	yes	no
7	182	8.0	5×	2.7	yes	yes	no
current	-	8.0	-	?	-	-	yes

- **Item properties selected by the current user:**
  - mpix = 8.0, waterproof="yes"
  - Nearest neighbors of current session: {1,3}
  - Recommendation: LCD-size = 2.5

Unsatisfied Requirements

"no solution could be found", e.g., price<=150 and sound=yes

Constraint relaxation

- relax constraints of a recommendation problem until a corresponding solution has been found

Users could also be interested in repair proposals

- recommender can calculate a solution by adapting the user-defined requirements

Unsatisfied Requirements: Example

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

customer requirements R =

{

r_1:price<=150,

r_2:opt-zoom=5x,

r_3:sound=yes,

r_4:waterproof=yes

}

No solution found:

$\sigma_{[R]}P = \{\}$

which requirements should be changed?

Conflict Sets: Example

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

$\sigma[r_1,r_2]P = \varnothing$

$\sigma[r_2,r_4]P = \varnothing$

$\sigma[r_1,r_3]P = \varnothing$

Conflict Sets and Diagnoses (1)

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

what are possible values?

Conflict Set

$CS=\{r_1, r_2, \ldots, r_k\} \subseteq R$ s.t. $\sigma[CS]P = \emptyset$

CS is minimal if not $\exists$ CS': CS' $\subset$ CS

[Junker 2004]

conflict sets=
{CS_1:{r_1,r_2},
CS_2:{r_2,r_4},
CS_3:{r_1,r_3}}

Diagnosis

$d_i \in D$ for (P, R): $d_i \subseteq R$ s.t. $\sigma[R-d_i]P \neq \emptyset$

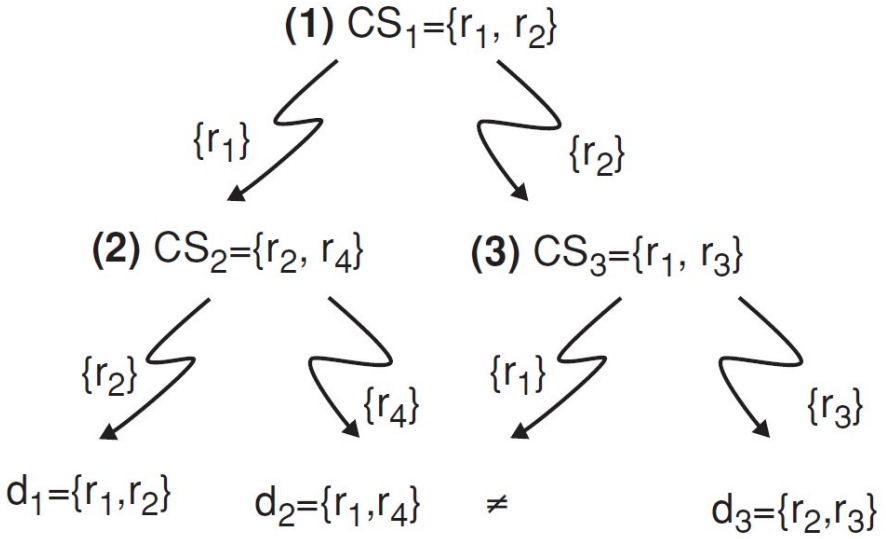
d_i min. not $\exists$ d_i': $d_i' \subset d_i$

[Reiter1987]

diagnoses D =
{d_1:{r_1,r_2},
d_2:{r_1,r_4},
d_3:{r_2,r_3}}

Conflict Sets and Diagnoses (2)

Calculate minimal diagnoses for unsatisfied requirements (HSDAG = Hitting Set Directed Acyclic Graph)



[Reiter1987]

The diagnoses derived from the conflict sets $\{CS_1, CS_2, CS_3\}$ are $\{d_1:\{r_1,r_2\}, d_2:\{r_1,r_4\}, d_3:\{r_2,r_3\}\}$

Determining Repair Actions: Example

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

π repair attributes

σ repair requirements

$$d=\{r_1,r_2\} \ldots \pi_{[r1:price,\ r2:opt\text{-}zoom]}\sigma_{[r3:sound=yes,\ r4:waterproof=yes]}P$$

$$rep_1 = \pi_{[attributes(d1)]}\sigma_{[R\text{-}d1]}P = \pi_{[r1:price,\ r2:opt\text{-}zoom]}\sigma_{[r3:sound=yes,\ r4:waterproof=yes]}P$$

$$rep_2 = \pi_{[attributes(d2)]}\sigma_{[R\text{-}d2]}P = \pi_{[r1:price,\ r4:waterproof]}\sigma_{[r2:opt\text{-}zoom=5x,\ r3:sound=yes]}P$$

$$rep_3 = \pi_{[attributes(d3)]}\sigma_{[R\text{-}d3]}P = \pi_{[r2:opt\text{-}zoom,\ r3:sound]}\sigma_{[r1:price <= 150,\ r4:waterproof=yes]}P$$

repair	price	opt-zoom	sound	waterproof
rep_1	r_1':price=278	r_2': opt-zoom=10x	r_3:sound=yes	r_4:waterproof=yes
rep_2	r_1':price=182	r_2:opt-zoom=5x	r_3:sound=yes	r_4':waterproof=no
rep_3	r_1:price=148	r_2': opt-zoom=4x	r_3':sound=no	r_4:waterproof=yes

Simple Conflict Detection: Algorithm

$R \leftarrow \{r_1, r_2, \ldots, r_n\}$; /* R: requirements*/
$B \leftarrow \emptyset$; /* B: constraints part of the minimal conflict */
IF inconsistent($\sigma_{[R]}P$) /* P: product assortment */
REPEAT
$\Delta \leftarrow B$; /* Δ: currently investigated constraint set */
REPEAT
$r \leftarrow$ element($R - \Delta$) /* r: one requirement */
$\Delta \leftarrow \Delta \cup \{r\}$;
UNTIL inconsistent($\sigma_{[\Delta]}P$)
$B \leftarrow B \cup \{r\}$;
UNTIL inconsistent($\sigma_{[B]}P$)
RETURN (B);

#consistency checks:
- worst case: $O(n^2/2+n)$
- best case: 3

Simple Conflict Detection: Example

$V=\{v_1,v_2,v_3\}$; $d_{v1}=d_{v2}=d_{v3}=[1..3]$; $\mathbf{R}=\{r_1,r_2,r_3\}$; $P = v_1 \times v_2 \times v_3$

$r_1: v_1 > v_2$; $r_2: v_2 > v_3$; $r_3: v_3 > v_1$

B: $\emptyset$; Δ: $\emptyset$

- r:r_1; Δ: $\{r_1\}$
- r:r_2; Δ: $\{r_1, r_2\}$
- r:r_3; Δ: $\{r_1, r_2, r_3\}$

B:$\{r_3\}$; Δ: $\{r_3\}$

- r:r_1; Δ: $\{r_3,r_1\}$
- r:r_2; Δ: $\{r_3,r_1,r_2\}$

B:$\{r_3,r_2\}$; Δ: $\{r_3,r_2\}$

- r:r_1; Δ: $\{r_3,r_2,r_1\}$

B:$\{r_3,r_2,r_1\}$ /* minimal conflict set */

Item Ranking

Multi-attribute Utility Theory (MAUT)

- each item is evaluated according to a predefined set of dimensions that provide an aggregated view on the basic item properties

E.g. quality and economy are dimensions in the domain of digital cameras

id	value	quality	economy
price	≤250	5	10
	>250	10	5
mpix	≤8	4	10
	>8	10	6
opt-zoom	≤9	6	9
	>9	10	6
LCD-size	≤2.7	6	10
	>2.7	9	5
movies	Yes	10	7
	no	3	10
sound	Yes	10	8
	no	7	10
waterproof	Yes	10	6
	no	8	10

Customer-specific Item Utilities with MAUT

Customer interests

customer	quality	economy
Cu_1	80%	20%
Cu_2	40%	60%

*

Item utilities:

**

quality	economy	utility: cu_1	utility: cu_2
P_1 Σ(5,4,6,6,3,7,10) = 41	Σ (10,10,9,10,10,10,6) = 65	45.8 [8]	55.4 [6]
P_2 Σ(5,4,6,6,10,10,8) = 49	Σ (10,10,9,10,7,8,10) = 64	52.0 [7]	58.0 **[1]**
P_3 Σ(5,4,10,6,10,10,8) = 53	Σ (10,10,6,10,7,8,10) = 61	54.6 [5]	57.8 [2]
P_4 Σ(5,10,10,6,10,7,10) = 58	Σ (10,6,6,10,7,10,6) = 55	57.4 [4]	56.2 [4]
P_5 Σ(5,4,6,10,10,10,8) = 53	Σ (10,10,9,6,7,8,10) = 60	54.4 [6]	57.2 [3]
P_6 Σ(5,10,6,9,10,10,8) = 58	Σ (10,6,9,5,7,8,10) = 55	57.4 [3]	56.2 [5]
P_7 Σ(10,10,6,9,10,10,8) = 63	Σ (5,6,9,5,7,8,10) = 50	60.4 [2]	55.2 [7]
P_8 Σ(10,10,10,9,10,10,10) = 69	Σ (5,6,6,5,7,8,6) = 43	63.8 **[1]**	53.4 [8]

Formula:

$$utility(p) = \sum_{j=1}^{\#(dimensions)} \underbrace{interest(j)}_{*} * \underbrace{contribution(p,j)}_{**}$$

Example Software: Investment Sales Support



wüstenrot

[Felfernig et al. 2006]

Case-based Recommender Systems

Search paradigms: query-based vs. navigation-based
identification of candidate items using similarity measures



Definition:

- sim (vp, vc) expresses for each item attribute value vp its distance to the customer requirement vc.
- w_{eight} is the importance weight for requirement vc.

In real world, customers would like to

- maximize certain properties. i.e. resolution of a camera, "more is better"(MIB)
- minimize certain properties. i.e. price of a camera, "less is better"(LIB)

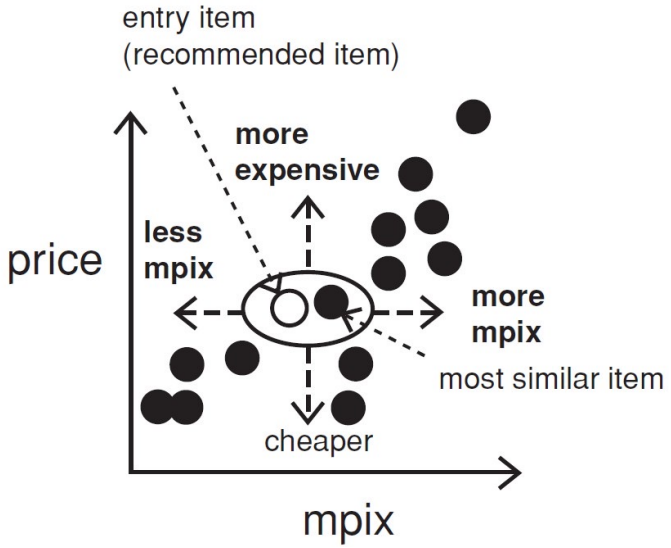
Similarity Metrics

id	Name	Formel
1	Nearer Is Better (NIB)	$sim(vc, vp) = 1 - \frac{\lvert vc - vp \rvert}{max(vp) - min(vp)}$
2	Equality is Better (EQ)	$vc = vp \rightarrow sim(vc, vp) = 1$ $vc \neq vp \rightarrow sim(vc, vp) = 0$
3	More Is Better (MIB)	$sim(vc, vp) = \frac{vc - min(vp)}{max(vp) - min(vp)}$
4	Less Is Better (LIB)	$sim(vc, vp) = \frac{max(vp) - vc}{max(vp) - min(vp)}$

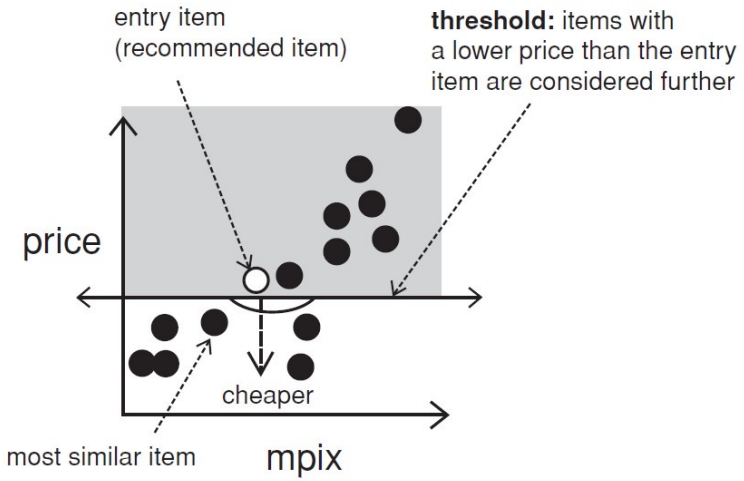
$$similarity(V, P) = \frac{\sum_{vi \in V} weight(vi) \times sim(vi, vp_i)}{\sum_{vi \in V} weight(vi)}$$

Interacting with Case-based Recommender Systems

- Customers maybe not know their preferences
- Critiquing is an effective way to support such navigations
- Customers specify their change requests (*price or mpix*) that are not satisfied by the current item (*entry item)*

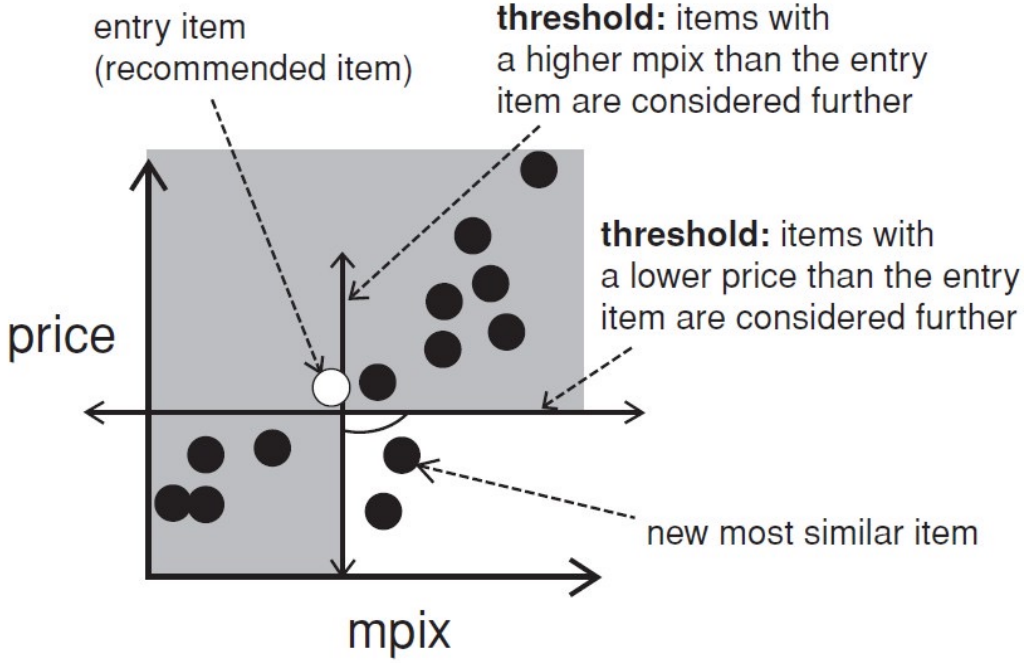


Critique on price



Case-based Recommender Systems: Compound Critiques

Operating over multiple properties can improve the efficiency of recommendation dialogs



Case-based Recommender: Example



- **Critiquing**
  - Similarity-based navigation
- **Unit critiques**
  - Critiquing of single properties
- **Compound critiques**
  - Critiquing of multiple properties
- **Dynamic critiques**
  - Critique options only available if applicable
  - Mining of frequent critique patterns
- **Incremental critiques**
  - Considers critiquing history
- **Experience-based critiques**
  - critique recommendation

References

A. Falkner, A. Felfernig, and A. Haag. Recommendation Technologies for Configurable Products, *AI Magazine*, AAAI, 32(3):99-108, 2011.

A. Felfernig, G. Friedrich, D. Jannach, and M. Zanker, An Integrated Environment for the Development of Knowledge-based Recommender Applications, *International Journal of Electronic Commerce* (*IJEC*), 11(2):11-34, 2006.

A. Felfernig, G. Friedrich, M. Schubert, M. Mandl, M. Mairitsch, and E. Teppan. Plausible Repairs for Inconsistent Requirements, 21st International Joint Conference on Artificial Intelligence (IJCAI'09), Pasadena, California, USA, pp. 791-796, 2009.

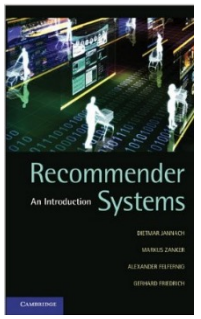
D. Jannach, M. Zanker, A. Felfernig, and G. Friedrich, Recommender Systems, Cambridge University Press, 2011.

U. Junker. QUICKXPLAIN: Preferred Explanations and Relaxations for Over-Constrained Problems. 19th National Conference on AI (AAAI'04), pp. 167—172, 2004.

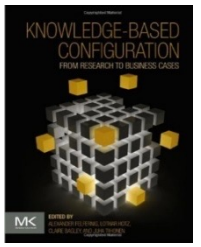
R. Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 23(1):57—95, 1987.

Course Material

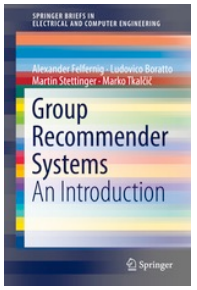
- D. Jannach, M. Zanker, A. Felfernig, and G. Friedrich. Recommender Systems – An Introduction, Cambridge University Press, 2010 (*can be found in library*).



- A. Felfernig, L. Hotz, C. Bagley, and J. Tiihonen. Knowledge-based Configuration – From Research to Business Cases, Elsevier/Morgan Kaufmann, 2014.



- Lecture slides and slides from recommenderbook.net.
- A. Felfernig, L. Boratto, M. Stettinger, and M. Tkalcic. Group Recommender Systems, Springer, 2018



Exercise 1

- Define a simple CSP for digital camera recommendation
  - X_I: variables describing the products (items) + D
  - X_U: variables describing the user model + D
  - KB: knowledge base with recommendation knowledge
  - R: provide an example of user requirements
  - I: product catalog (set of offered items)

id	price(€)	mpix	opt-zoom	LCD-size	movies	sound	waterproof
P_1	148	8.0	4×	2.5	no	no	yes
P_2	182	8.0	5×	2.7	yes	yes	no
P_3	189	8.0	10×	2.5	yes	yes	no
P_4	196	10.0	12×	2.7	yes	no	yes
P_5	151	7.1	3×	3.0	yes	yes	no
P_6	199	9.0	3×	3.0	yes	yes	no
P_7	259	10.0	3×	3.0	yes	yes	no
P_8	278	9.1	10×	3.0	yes	yes	yes

- Note: no submission („Abgabe“) needed!

Exercise 2

- Determine one minimal conflict set for the following CSP (on the basis of “Simple Conflict Detection”)
  - $V=\{v_1,v_2,v_3\}$; $d_{v1}=d_{v2}=d_{v3}=[1..3]$; $R=\{r_1,r_2,r_3\}$; $P = v_1 \times v_2 \times v_3$
  - $r_1: v_1 > v_2$; $r_2: v_2 = v_1$; $r_3: v_3 <> v_1$
- Determine all minimal conflict sets for the following CSP (on the basis of “Simple Conflict Detection”)
  - $V=\{v_1,v_2,v_3\}$; $d_{v1}=d_{v2}=d_{v3}=[1..3]$; $R=\{r_1,r_2,r_3\}$; $P = v_1 \times v_2 \times v_3$
  - $r_1: v_1 > v_2$; $r_2: v_2 > v_2$; $r_3: v_3 <> v_1$
- Note: no submission („Abgabe“) needed!

Exercise 3

- Determine the most similar item (product = loan) from the item list on the next slide (with regard to the following set of customer requirements).
  (define on your own the weights of the dimensions)

Product attribute	Value	Similarity Metric
current-creditworthiness (ccw)	3	EQ
intended-loansum (ils)	6000	NIB
intended-runtime (irt)	6 years	NIB

- Note: no submission („Abgabe“) needed!

Exercise 3

Produktattribut	loan1	loan2	loan3	loan4	loan5	loan6	loan7	loan8	loan9
loan-id	1	2	3	4	5	6	7	8	9
currency	EUR	EUR	EUR	EUR	EUR	EUR	EUR	EUR	EUR
creditworthiness	1	2	3	1	2	3	1	2	3
loansum	1.000	10.000	10.000	40.000	250.000	350.000	150.000	100.000	200.000
periodicalpayment	1.040	3.667	2.875	7.600	15.200	22.190	19.950	9.067	13.200
runtime	1	3	4	6	25	25	10	15	25
interestrate	4%	5%	6%	4%	4%	4.5%	6%	4.5%	5%
cadastreentry	n	n	n	n	j	j	n	n	j
numberofdebtors	1	1	2	1	1	1	1	2	1
residualdebt	0	0	0	0	0	0	5000	0	0

Thank You!